

Prom π t: Engineering Strategy & Development Guardrails

Senior Architecture Review

June 16, 2026

Strategic Overview

Building an IDE requires managing immense complexity, high-frequency state updates, and complex background processes. This document outlines the core strategic principles to ensure **Prom π t** transitions from a theoretical architecture to a production-grade, highly performant Electron application.

1 Phase-Based Development Strategy

Attempting to build the AI, the terminal, the file tree, and the Language Server simultaneously will result in a fragmented codebase. Adhere to a strict, phased approach:

1. **Phase 1 (The Shell):** Establish the Electron/Vite build pipeline. Render a basic React interface to confirm process initialization.
2. **Phase 2 (The Canvas):** Integrate Monaco Editor as an uncontrolled React component. Verify layout recalculations upon window resizing.
3. **Phase 3 (The Bridge):** Establish the secure `preload.js` IPC bridge. Validate bi-directional communication (e.g., reading a local file via `Node.js` and rendering it in Monaco).
4. **Phase 4 (The AI Simulation):** Implement UI streaming functionality using a mock backend generator before integrating paid LLM APIs. Ensure token batching prevents main-thread blocking.

2 Critical Subsystem Guardrails

2.1 File System Synchronization

A static file tree is insufficient.

- **Requirement:** Utilize the `chokidar` library in the Main Process to watch workspace directories.
- **Safety Mechanism:** You *must* debounce file-system events. Operations like `npm install` trigger thousands of events; failing to debounce will overwhelm the IPC bridge and crash the React renderer.

2.2 State Management

High-frequency UI updates require granular state control.

- **Requirement:** Avoid heavy global stores like Redux. Utilize atomic state management solutions such as Zustand or Jotai.
- **Goal:** Ensure that updating a minor UI element (e.g., an unsaved file indicator) does not trigger a re-render of the entire sidebar or editor wrapper.

2.3 Language Server Provisioning

Implementing Path B (LSP) requires ensuring the user's OS has the necessary background binaries.

- **Strategy:** Do not bundle massive binaries into the executable.
- **Implementation:** Upon opening a file (e.g., Python), check the system PATH. If the server (`pylsp`) is missing, trigger a native UI notification offering to automatically install it via background terminal commands (`pip install`).

3 AI Integration Polish

The efficacy of an AI editor relies heavily on invisible context gathering.

- **Context Assembly:** Never forward raw user prompts directly to the LLM.
- **System Prompting:** Silently assemble a master prompt containing the operating system, active filename, language ID, and the text buffer immediately surrounding the cursor.
- **Streaming Optimization:** Batch incoming AI tokens into a buffer and flush to the Monaco text model at 16ms intervals (`requestAnimationFrame`) to maintain 60FPS scrolling.

4 The Hallmark of Production Software

The distinction between an academic project and production software is **graceful error handling**. If the Node backend fails an operation, it must not fail silently into a hidden terminal console. Catch the error, transmit it via IPC, and render a polished, actionable error notification within the React UI.